

UNIVERSIDAD TECNOLÓGICA NACIONAL (UTN)

Extensión Áulica Necochea

METODOLOGÍA DE SISTEMAS 2

Trabajo Final Integrador

*Portal Web de la Asociación Cooperadora del Hospital
Municipal "Dr. Emilio Ferreyra" (Necochea)*

MIEMBROS DEL EQUIPO Y ROLES:

Thiago Masson	Full Stack Developer & Branding Specialist
Kevin Nielsen	Security & Backend Developer
Aramis Prieto	Scrum Master & Lead Backend Developer
Santiago Ialungo	UI/UX & Lead Frontend Developer

Docente: Daniel Moreno

Fecha de Entrega: Junio 2026 · Necochea, Argentina

1. Descripción General del Alcance del Proyecto

El proyecto consiste en el diseño, desarrollo e implementación de un portal web interactivo, robusto y con arquitectura en la nube para la Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra" de la ciudad de Necochea. El alcance abarca la digitalización de los procesos tradicionales de la institución, reemplazando las planillas de papel por un sistema auditable.

Requerimientos generales planteados:

- **Gestión y Autogestión de Socios:** Registro en línea de usuarios, con un panel privado donde cada socio puede completar sus datos obligatorios (DNI único, domicilio, teléfono) para ser incorporado formalmente al Libro Registro de Asociados digital.
- **Módulo Transaccional de Cuotas Sociales:** Historial de pagos mensuales, control de estados de membresía y pasarela de pago recurrente mediante Mercado Pago.
- **Campañas de Recaudación Transparentes:** Visualización pública de campañas de recaudación activas para la compra de insumos médicos o aparatología, provistas de barras de progreso financiero calculadas en tiempo real frente a metas objetivas.
- **Declaración y Auditoría de Donaciones:** Checkout interactivo para registrar transferencias bancarias manuales mediante la carga de comprobantes, y un sistema alternativo de donación directa en línea a través de Mercado Pago.
- **Panel Administrativo:** Interfaz de control exclusiva para operadores que permite aprobar o rechazar transferencias, dar de alta/baja/modificar campañas, y gestionar el estado del padrón de socios.
- **Módulo Informativo de Actualidad:** Portal dinámico de publicación de novedades y logros institucionales.

2. Miembros del Equipo y Roles Asignados

Para la planificación, desarrollo y optimización del portal web, se definieron roles basados en las fortalezas individuales de los integrantes, asegurando una metodología de trabajo ágil y colaborativa:

- **Aramis Prieto (Scrum Master & Lead Backend Developer):** Responsable de facilitar ceremonias ágiles, orquestar los modelos e interconexión de bases de datos híbridas, controlar la transaccionalidad concurrente mediante bloqueos de fila relacionales y programar la suite de pruebas de integración automatizadas.
- **Kevin Nielsen (Security & Backend Developer):** Encargado de implementar la autenticación segura por JSON Web Tokens (JWT), configurar

middlewares de control de tasa de solicitudes (Rate Limiting) por IP, estructurar validaciones mediante expresiones regulares y desarrollar el servicio de correos automatizados SMTP.

- **Santiago Ialungo (UI/UX & Lead Frontend Developer):** Responsable del diseño visual adaptado al entorno de salud (paleta clínica, fondos ECG), maquetación responsiva con Tailwind CSS de las vistas públicas y el panel administrativo, e integración de Lenis scroll y Navbar Scroll-Spy.
- **Thiago Masson (Full Stack Developer & Branding Specialist):** Encargado de la migración y estandarización del monorrepo mediante entornos pnpm, integración de logotipos oficiales de la cooperadora, desarrollo del módulo documental de noticias e implementación de capas de sanitización contra ataques XSS con DOMPurify.

3. Descripción de la Arquitectura Seleccionada

Se ha optado por una Arquitectura Cliente-Servidor Desacoplada apoyada en una Persistencia Híbrida para responder de manera óptima a las necesidades transaccionales y multimedia del portal.

- **Lenguaje y Entorno:** JavaScript/Ecosystem de extremo a extremo, utilizando Node.js (Express) en el backend y React.js (Vite) en el frontend.
- **Frameworks y Librerías Base:** Sequelize como ORM para la interacción relacional y Mongoose como ODM para el motor documental.
- **Repositorio y Monorrepo:** Alojado en GitHub, orquestado mediante workspaces de pnpm para agilizar la consistencia de dependencias entre el cliente y el servidor.

Arquitectura Web / Datos:

1. **Motor Relacional (SQL - PostgreSQL / MySQL):** Resguarda la información con estrictas garantías ACID. Contiene las tablas de usuarios (credenciales con claves hashadas mediante bcryptjs), perfiles_socios (datos registrales) y campanas_eco (totales financieros y metas de recaudación).²
2. **Motor Documental (NoSQL - MongoDB Atlas):** Almacena estructuras dinámicas de formato libre y alta carga multimedia. Contiene las colecciones de noticias_actualidad y campanas_detalle (narrativas enriquecidas, galerías fotográficas y testimonios) vinculadas lógicamente al ID relacional.
3. **Data Mashup:** Para reducir latencias de red, el backend ejecuta consultas en paralelo en ambos motores mediante Promise.all, unificando la respuesta financiera (SQL) y descriptiva (NoSQL) en un solo objeto JSON plano enviado al cliente en una única petición.

4. Análisis del Estado Actual del Código y Propuestas de Mejora

A continuación, se detalla el estado actual del código y se detectan bloques de oportunidad para incorporar buenas prácticas mediante patrones de diseño más simples y directos.

Patrón Singleton (Creacional)

- Módulo / paquete: backend/config/db.js (Conexión a la base de datos).
- Solución a implementar: Implementar el patrón Singleton para la clase de conexión a la base de datos PostgreSQL/MongoDB. Asegura que toda la aplicación comparta una única instancia de conexión.
- Ventajas de la implementación: Ahorro de memoria, previene la sobrecarga del pool de conexiones al evitar múltiples instancias innecesarias.
- Posibles desventajas: Introduce estado global en la aplicación, lo que puede dificultar los tests unitarios si no se maneja correctamente el mock de la conexión.

Patrón Factory Method (Creacional)

- Módulo / paquete: backend/services/userService.js (Creación de usuarios).
- Solución a implementar: Utilizar una función o clase fábrica que, dado el rol (Socio o Admin), devuelva la instancia correcta del modelo de usuario con sus respectivos permisos precargados.
- Ventajas de la implementación: Desacopla la lógica de creación de objetos de los controladores, cumpliendo el principio Open/Closed al poder añadir nuevos tipos de usuario fácilmente.
- Posibles desventajas: Puede generar una sobreingeniería si los tipos de usuario son muy estáticos y no requieren procesos de creación distintos.

Patrón Facade (Estructural)

- Módulo / paquete: backend/controllers/donacionController.js (Orquestación de donaciones).
- Solución a implementar: Crear un 'DonacionFacade' que exponga un método unificado (ej: procesarDonacion). Este Facade se encargará internamente de validar al usuario, interactuar con el modelo de pagos y llamar al servicio de email.
- Ventajas de la implementación: Limpia el controlador HTTP, escondiendo la complejidad técnica detrás de una interfaz simple y fácil de leer.

- Posibles desventajas: El Facade corre el riesgo de convertirse en un 'Objeto Dios' (God Object) si asume más responsabilidades de las que debería, rompiendo la cohesión.